

Trabalho Prático — Desenvolvimento de Software

Entrega Parcial (Fase 1)

Johann Hardt & Rafael Lucht

Conteúdo

1	Requisitos	2
1.1	Configurações iniciais	2
1.2	Interações com o usuário	2
1.3	Conversão de texto e reprodução de áudio	2
2	Interface com o usuário (GUI)	3
2.1	Descrição geral do leiaute	3
2.1.1	Seção superior	3
2.1.2	Seção à esquerda	3
2.1.3	Seção à direita	3
2.1.4	Seção inferior	3
2.2	Croqui	4
3	Classes	4
3.1	AudioParameters	4
3.2	TextToEventMapper	4
3.3	AudioEvent	5
3.4	NoteEvent	5
3.5	InstrumentChangeEvent	5
3.6	OctaveChangeEvent	5
3.7	SilenceEvent	5
3.8	Score	5
3.9	PlaybackContext	6
3.10	AudioPlayer	6
3.11	TextInput	6
3.12	MainController	6
3.13	AudioEngine	7

1 Requisitos

Abaixo nesta seção estão listadas as funcionalidades de que o aplicativo deverá dispor.

1.1 Configurações iniciais

O sistema deverá ser inicializado com valores pré-definidos dos seguintes parâmetros:

- instrumento inicial;
- oitava inicial;
- volume;
- BPM;

os quais poderão ser alterados pelo usuário.

1.2 Interações com o usuário

O sistema deverá permitir que o usuário:

- forneça uma entrada de texto, via teclado ou um arquivo “.txt”;
- redefina os parâmetros padrão de áudio (instrumento e oitava iniciais);
- ajuste a qualquer momento as configurações de áudio (volume e BPM) via elementos interativos;
- consulte à demanda o mapeamento “caractere $\mapsto$ evento”;
- restaure as configurações padrão do sistema;
- desencadeie a reprodução de áudio quando houver um texto disponível;
- pause/retome, reinicie ou aborte a reprodução de áudio em qualquer instante desta.

1.3 Conversão de texto e reprodução de áudio

Após receber o texto do usuário, o sistema deverá:

- interpretar o texto, aplicando o mapeamento de caracteres para lê-lo como uma partitura;
- quando solicitado, reproduzir áudio conforme o texto fornecido e as configurações de áudio;
- exibir na interface o próximo elemento a ser lido.

2 Interface com o usuário (GUI)

2.1 Descrição geral do leiaute

A janela de interface será composta por 4 seções: superior, esquerda, direita, e inferior. Seus leiautes estão descritos abaixo.

2.1.1 Seção superior

O *topbar*, que deverá dispor de:

- um nome para o aplicativo;
- uma opção de consulta ao mapeamento “caractere $\mapsto$ evento”.

Ao solicitar uma consulta ao mapeamento de caracteres, uma outra interface será sobreposta à principal; nessa será listado o mapeamento definido.

2.1.2 Seção à esquerda

A seção à esquerda será dedicada às entradas de texto do usuário. Nela, deverá haver:

- um campo em que o usuário inserirá o texto livre;
- uma descrição do espaço de texto sendo usado — [n.^o de car. escritos]/[n.^o de car. limite];
- uma opção para carregar um “.txt” como texto fonte, próximo ao campo de texto;
- uma opção para apagar todo o texto no campo de escrita;

2.1.3 Seção à direita

Na seção à direita estarão as configurações do sistema, como:

- uma barra horizontal que representa o volume, com controle deslizante;
- um campo que representa BPM numericamente, sujeito a alterações provenientes do usuário;
- um menu *dropdown* de seleção do instrumento inicial;
- um menu *dropdown* de seleção da oitava inicial;
- uma grade de seleção de plano de fundo — o “tema” visual;
- um botão de “restaurar padrão”, que reverterá as configurações às iniciais padrão do sistema.

2.1.4 Seção inferior

A seção inferior será a *playbar*, que lidará com os recursos de reprodução de áudio. Tais recursos incluem:

- um botão de *play*, que alternará entre “pausar” e “retomar” conforme o estado atual;
- um botão de *stop*, que acabará com o processo de reprodução de áudio atual;
- uma barra de progresso da leitura do texto;
- o próximo caractere a ser lido, junto ao “evento” que este desencadeia;
- uma opção para exportar o texto como arquivo “.txt”.

2.2 Croqui

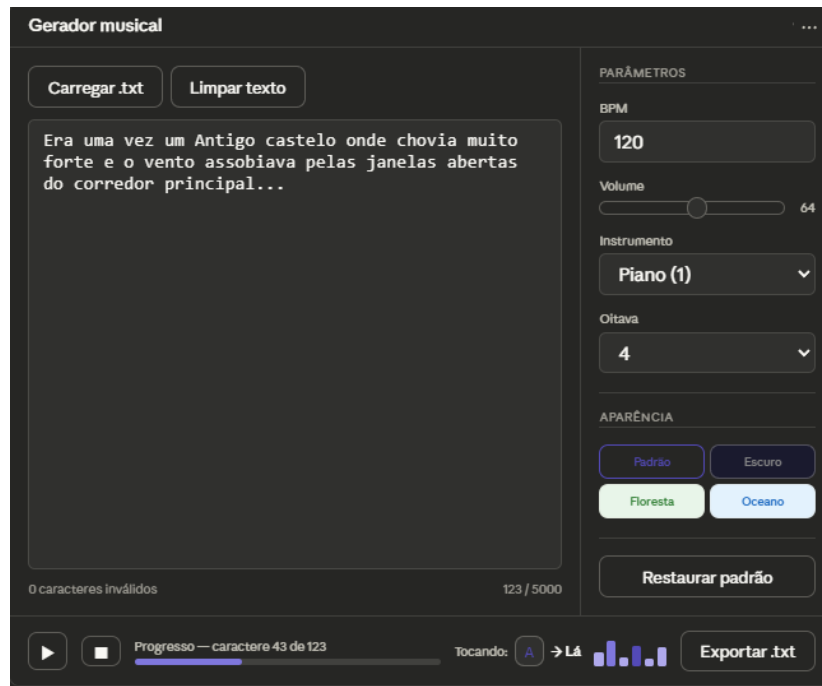


Figura 1: Ilustração do leiaute conceitual

3 Classes

O projeto será desenhado de acordo com os paradigmas de *Object-Oriented Programming* (OOP), baseado em classes e objetos que definem atributos e métodos, seguindo também os princípios SOLID. Assim, o projeto visa um desenho de *software* modular, extensível e de fácil manutenção.

As classes do projeto deverão ser internamente coerentes, independentes e compatíveis entre si. Isso permitirá um sistema de classes que configura arranjos complexos de módulos pequenos, os quais podem ser submetidos a testes unitários para averiguar sua corretude de forma limpa e certa.

Abaixo estão as classes pretendidas, com responsabilidades distintas.

3.1 AudioParameters

Esta classe é composta pelas classes “básicas”:

- AudioVolume;
- AudioBPM;
- AudioInstrument;
- AudioOctave.

Todas dispõe de métodos *get/set* e *reset*, com atributos numéricos (volume e BPM) ou classe-enumerados (instrumento e oitava).

3.2 TextToEventMapper

Esta classe é responsável por converter caracteres de entrada em “eventos” executáveis.

Para atributos, é composta por um mapeamento entre grupos de caracteres e classes de eventos. Dispõe de métodos para:

- usar o mapeamento para a conversão “caractere $\mapsto$ evento”;

- consultar os elementos do mapa.

Cada caractere está associado a uma classe de *AudioEvent*.

3.3 AudioEvent

Esta é uma interface que representa ações relacionadas ao áudio, como:

- NoteEvent;
- InstrumentChangeEvent;
- OctaveChangeEvent;
- SilenceEvent.

Cada classe implementa um método de execução de evento em um contexto de reprodução.

3.4 NoteEvent

Esta classe contém métodos para a execução de uma nota musical.

É composta somente por *Note* (do, re, mi...).

Ao ser executada, aciona a reprodução de uma nota conforme os parâmetros atuais de áudio.

3.5 InstrumentChangeEvent

Esta classe representa a mudança do instrumento atual.

É composta por *DstInstrument*.

Ao ser executada, atualiza o instrumento no contexto de reprodução.

3.6 OctaveChangeEvent

Esta classe representa a modificação da oitava atual.

É composta somente por *DstOctave*.

Ao ser executada, atualiza a oitava no contexto de reprodução.

3.7 SilenceEvent

Esta classe representa uma pausa temporal na reprodução.

É composta por *duration*.

Ao ser executada, introduz um intervalo sem geração de som.

3.8 Score

Esta classe representa uma sequência de eventos de áudio derivados de um texto.

É composta por:

- uma coleção ordenada de objetos AudioEvent;
- um índice da posição atual.

Dispõe de métodos para:

- acessar o próximo evento;
- reiniciar a posição de leitura;
- verificar se a sequência foi concluída.

3.9 PlaybackContext

Esta classe encapsula o estado atual da execução de áudio.

É composta por:

- AudioParameters;
- uma referência a uma interface de áudio.

Fornece acesso controlado a:

- os parâmetros atuais de reprodução;
- operações de saída de áudio.

3.10 AudioPlayer

Esta classe é responsável por gerenciar a execução de uma Score.

É composta por:

- uma Score;
- um PlaybackContext;
- um estado de reprodução (playing, paused, stopped);

Dispõe de métodos para:

- iniciar a reprodução;
- pausar e retomar;
- parar a reprodução;
- avançar na sequência de eventos;

3.11 TextInput

Esta classe representa o texto fornecido pelo usuário.

É composta somente por um buffer de string, e dispõe de métodos para:

- definir ou atualizar o texto;
- carregar texto a partir de arquivo;
- limpar o conteúdo atual.

3.12 MainController

Esta classe coordena a interação entre entrada de texto, mapeamento e reprodução.

É composta por:

- TextInput;
- TextMapper;
- Score;
- AudioPlayer.

Dispõe de métodos para:

- processar o texto em uma sequência de eventos;
- controlar a reprodução (play, pause, stop);
- atualizar os parâmetros de áudio.

3.13 AudioEngine

Esta interface abstrai o sistema de geração de áudio.

Dispõe de métodos virtuais para:

- reproduzir uma nota;
- gerar silêncio por uma duração determinada.